

Kubernetes ConfigMap & Secret

configuring applications · Kubernetes in Action · Chapter 7

minikube · ConfigMap · Secret · configMap volume · TLS

오늘 다룰 내용

#	주제	핵심
1	왜 설정을 분리하는가	환경별 YAML 복사 문제
2	command / args	Docker ENTRYPOINT / CMD 대응
3	env (환경 변수)	OS의 NAME=value 주입
4	ConfigMap	키-값 맵 + 3가지 주입 경로
5	configMap 볼륨	파일 트리로 마운트, 갱신 동작
6	Secret	민감 데이터, TLS, ConfigMap과의 차이

7장은 "같은 Pod 매니페스트로 환경마다 다른 설정" 을 만드는 법

1. 왜 설정을 분리하는가

"같은 Pod로 dev/stage/prod"

문제: YAML 안에 값을 박으면

```
env:  
- name: INTERVAL  
  value: "30"      # 운영은 "5", 스테이징은 "60"...
```

- 환경마다 **YAML** 사본이 늘어남 → 드리프트, 리뷰 부담
- 비밀번호·키를 평문으로 매니페스트·Git에 두는 건 위험
- 컨테이너 이미지 안에 굽는 것도 같은 문제 + 재빌드

해결: 값과 매니페스트를 분리 → ConfigMap / Secret

컨테이너 설정의 4가지 경로

경로	어디에 적나	언제 좋은가
<code>command</code> / <code>args</code>	Pod 컨테이너 스펙	시작 명령·인자 변경
<code>env</code> (환경 변수)	Pod 컨테이너 스펙	짧은 값, 앱이 이미 env 사용
ConfigMap	클러스터 리소스	환경별로 다른 설정·파일
Secret	클러스터 리소스	비밀번호·키·인증서

ConfigMap·Secret은 다시 **env** 또는 **볼륨(파일)** 으로 Pod에 들어옴

2. command / args

Dockerfile의 ENTRYPOINT / CMD를 Pod에서 덮어쓰기

ENTRYPOINT / CMD ↔ command / args

Docker	Kubernetes	의미
ENTRYPOINT	command	실행 파일 (보통 그대로 둠)
CMD	args	인자 (자주 바꿈)

```
spec:
  containers:
  - image: luksa/fortune:args
    args: ["2"] # 이미지 CMD를 덮어쓰기 (2초 간격)
```

- **exec form** (ENTRYPOINT ["node", "app.js"])이 PID 1 시그널 처리에 안전
- **command** / **args** / **env** 는 Pod 생성 후 in-place 변경 불가 → 재생성

실습 1 — args 로 컨테이너 인자 덮어쓰기

```
spec:
  containers:
  - image: luksa/fortune:args
    args: ["2"]           # 이미지 CMD를 덮어쓰기
    name: html-generator
    volumeMounts: [{ name: html, mountPath: /var/htdocs }]
  - image: nginx:alpine
    name: web-server
    volumeMounts: [{ name: html, mountPath: /usr/share/nginx/html, readOnly: true }]
  volumes: [{ name: html, emptyDir: {} }]
```

```
$ kubectl apply -f fortune-pod-args.yaml && kubectl wait --for=condition=Ready pod/fortune2s
$ kubectl logs fortune2s -c html-generator --tail=2
Configured to generate new fortune every 2 seconds
Sat May 9 15:16:11 UTC 2026 Writing fortune to /var/htdocs/index.html
$ kubectl get pod fortune2s
```

NAME	READY	STATUS	RESTARTS	AGE
fortune2s	2/2	Running	0	1s

3. 환경 변수 (env)

OS가 프로세스에게 주는 NAME=value

env: 컨테이너 OS에 주입

```
env:  
- name: INTERVAL  
  value: "30"  
- name: MESSAGE  
  value: "간격은 ${INTERVAL}초"    # 앞 변수 참조
```

- 컨테이너에서 `printenv INTERVAL` → `30`
- `$(VAR)` 치환은 같은 컨테이너 env 목록 안에서만 (위→아래)
- Pod 단위 env는 없음 — 컨테이너마다 따로 적음
- 단점: 값을 YAML에 박으면 환경별 사본 문제 → ConfigMap으로 분리

실습 2 — env 로 환경 변수 직접 주입

```
$ kubectl apply -f fortune-pod-env.yaml
$ kubectl wait --for=condition=Ready pod/fortune-env --timeout=180s
pod/fortune-env condition met

$ kubectl exec fortune-env -c html-generator -- printenv INTERVAL
30

$ kubectl logs fortune-env -c html-generator --tail=1
Configured to generate new fortune every 30 seconds
```

- Pod YAML의 `env: [{name: INTERVAL, value: "30"}]` 이 컨테이너 OS에 주입
- 값은 여전히 YAML에 박힌 상태 → 다음 단계에서 ConfigMap으로 분리

4. ConfigMap

키-값 저장소 + Pod에 붙이는 3가지 경로

ConfigMap이란

```
apiVersion: v1
kind: ConfigMap
metadata: { name: fortune-config }
data:
  sleep-interval: "25"           # 짧은 값
  my-nginx-config.conf: |       # 여러 줄 설정 파일
    server { listen 80; gzip on; ... }
```

- 네임스페이스 스코프 UTF-8 키-값 맵
- 값은 짧은 문자열부터 여러 줄 설정 파일 전체까지
- API 객체는 etcd에 저장 → kubelet이 Pod 기동 시 읽어 주입

ConfigMap 만들기

```
# 리터럴
kubectl create configmap fortune-config --from-literal=sleep-interval=25

# 파일 하나 (키 = 파일명)
kubectl create configmap nginx-config --from-file=my-nginx-config.conf

# 키 이름 직접 지정
kubectl create configmap nginx-config --from-file=server.conf=https-nginx.conf

# 디렉터리 통째로 (파일마다 키 하나)
kubectl create configmap fortune-config --from-file=configmap-files/

# YAML 파일로
kubectl apply -f fortune-config.yaml
```

Pod에 붙이는 3가지 경로

경로	필드	쓰임
키 1개 → env	<code>valueFrom.configMapKeyRef</code>	짧은 값 한 개
맵 전체 → env	<code>envFrom.configMapRef</code> (+ <code>prefix</code>)	여러 키를 env로 일괄
파일 트리 → 볼륨	<code>volumes.configMap</code>	nginx 등 파일 기반 설정

한 키 → env에 필요한 3가지

1. 컨테이너의 env 이름 — `env[].name` (예: `INTERVAL`)
2. ConfigMap 리소스 이름 — `configMapKeyRef.name` (예: `fortune-config`)
3. 그 안의 키 이름 — `configMapKeyRef.key` (예: `sleep-interval`)

실습 3 — ConfigMap 값을 env 로 받기 (valueFrom)

```
# fortune-config.yaml
data:
  sleep-interval: "25"

# fortune-pod-env-configmap.yaml
env:
- name: INTERVAL
  valueFrom:
    configMapKeyRef:
      name: fortune-config
      key: sleep-interval
```

```
$ kubectl apply -f fortune-config.yaml -f fortune-pod-env-configmap.yaml
$ kubectl get configmap fortune-config -o jsonpath='{.data.sleep-interval}'
25
$ kubectl exec fortune-env-from-configmap -c html-generator -- printenv INTERVAL
25
```

Pod YAML에 25가 없는데 컨테이너에는 25 ← kubelet이 ConfigMap 조회 후 주입

실습 4 — ConfigMap → env → args 치환 (`$(VAR)`)

```
spec:
  containers:
  - image: luksa/fortune:args          # 이미지가 인자만 받음 (env 안 읽음)
    env:
      - name: INTERVAL
        valueFrom: { configMapKeyRef: { name: fortune-config, key: sleep-interval } }
    args: ["$(INTERVAL)"]              # env를 인자에 끼워 넣기
```

```
$ kubectl logs fortune-args-from-configmap -c html-generator --tail=1
Configured to generate new fortune every 25 seconds
```

- `args` 는 ConfigMap 직접 참조 불가 → **env로 한 번 거쳐 `$(VAR)` 치환**

5. configMap 볼륨

ConfigMap을 파일 트리로 마운트

언제 볼륨을 쓰는가

상황	권장
짧은 플래그 1~2개	env
ConfigMap의 여러 키를 env 일괄	envFrom
nginx / Java properties / prometheus.yml	볼륨 (파일)
TLS 키 / 인증서	Secret 볼륨

```
volumes:
- name: config
  configMap: { name: fortune-config }
volumeMounts:
- { name: config, mountPath: /etc/nginx/conf.d, readOnly: true }
```

각 ConfigMap 키 = `mountPath` 아래 파일 하나

실습 5 — 디렉터리로 ConfigMap 만들고 볼륨으로 마운트

```
$ ls configmap-files/  
my-nginx-config.conf  sleep-interval  
  
$ kubectl create configmap fortune-config --from-file=configmap-files/  
$ kubectl apply -f fortune-pod-configmap-volume.yaml  
  
$ kubectl exec fortune-configmap-volume -c web-server -- ls /etc/nginx/conf.d  
my-nginx-config.conf  
sleep-interval
```

- `configmap-files/` 안의 두 파일이 컨테이너 안 두 파일로 마운트
- 디렉터리 마운트 시 원래 디렉터리 파일은 가려짐 → nginx의 `conf.d` 같은 보조 디렉터리를 사용

실습 6 — 응답 헤더로 nginx 설정 적용 검증 (gzip)

```
$ (kubectl port-forward pod/fortune-configmap-volume 28080:80 &)  
$ curl -sI -H "Accept-Encoding: gzip" http://127.0.0.1:28080/ | head  
HTTP/1.1 200 OK  
Server: nginx/1.29.8  
Content-Encoding: gzip           # ← ConfigMap의 gzip on; 이 적용됨
```

ConfigMap의 nginx 설정 → `/etc/nginx/conf.d/*.conf` 자동 include → nginx 동작 변경 → HTTP 응답에 반영

items / defaultMode

```
volumes:
- name: config
  configMap:
    name: fortune-config
    items:                                # 일부 키만, 다른 파일명으로
      - key: my-nginx-config.conf
        path: gzip.conf
    defaultMode: 0644                    # 파일 권한
```

- **items**: 같은 ConfigMap에서 컨테이너별로 노출할 키만 선택
- **defaultMode**: 마운트 파일 권한 비트
- **subPath** (한 파일만 마운트): 갱신이 안 따라올 수 있음 → 디렉터리 마운트 권장

실습 7 — items 로 일부 키만 노출

```
$ kubectl apply -f fortune-pod-configmap-volume-with-items.yaml
$ kubectl exec fortune-configmap-volume-with-items -c web-server -- ls -la /etc/nginx/conf.d
total 0
drwxr-xr-x 2 root root 23 May  9 15:24 ..2026_05_09_15_24_42.3692742159
lrwxrwxrwx 1 root root 32 May  9 15:24 ..data -> ..2026_05_09_15_24_42.3692742159
lrwxrwxrwx 1 root root 16 May  9 15:24 gzip.conf -> ..data/gzip.conf
```

- `sleep-interval` 은 노출되지 않음 (items에 없으니까)
- `..data` → `..timestamp_dir` 이중 심링크 = ConfigMap 갱신 시 원자적 교체용

갱신과 핫 리로드

단계	즉시 반영?
API/etcd의 ConfigMap	✅ 즉시
Pod 안 마운트 파일	⌚ 수십 초 ~ 약 1분
앱이 새 설정을 사용	❌ 보통 명시적 reload 필요

```
kubectl exec POD -c web-server -- nginx -s reload
```

subPath 단일 파일 마운트는 갱신이 안 될 수 있음

실습 8 — ConfigMap 갱신: 실측한 전파 지연

```
# 1) 로컬에서 gzip on → off 패치
$ sed -i '' 's/gzip on/gzip off/' /tmp/my-nginx-config-patched.conf

# 2) 같은 이름으로 ConfigMap 재생성
$ kubectl delete configmap fortune-config && kubectl create configmap fortune-config \
  --from-file=my-nginx-config.conf=/tmp/my-nginx-config-patched.conf ...

# 3) API에는 즉시 반영
$ kubectl get configmap fortune-config -o jsonpath='{.data.my-nginx-config\.conf}' | grep gzip
gzip off;                                # ← API에는 off

# 4) 그러나 20초 후에도 Pod 안 파일은 옛 내용
$ kubectl exec POD -c web-server -- grep gzip /etc/nginx/conf.d/my-nginx-config.conf
gzip on;                                # ← 마운트 파일은 아직 on

$ curl -sI -H "Accept-Encoding: gzip" http://127.0.0.1:28081/ | grep -i Encoding
Content-Encoding: gzip                  # ← 응답도 gzip 그대로
```

시간이 더 흐르거나 Pod 재시작 → 새 내용으로 반영

6. Secret

민감 데이터 — TLS, 비밀번호, 토큰

ConfigMap vs Secret — 같은 듯 다른 두 객체

비교 항목	ConfigMap	Secret
용도	일반 설정 (공개돼도 OK)	민감 데이터 (패스워드·키·토큰)
API 표현	<code>data</code> 평문 텍스트	<code>data</code> Base64 (인코딩 ≠ 암호화)
평문 입력	<code>data</code> 그대로	<code>stringData</code> (쓰기 전용 편의 필드)
바이너리	어색함	자연스러움 (TLS 키 등)
크기 한도	실무적으로 작게	약 1MB
타입(<code>type</code>)	단일	<code>Opaque</code> / <code>kubernetes.io/tls</code> / <code>dockerconfigjson</code> / <code>service-account-token</code>
etcd 암호화	일반 객체	at rest 암호화 옵션 권장
노드 저장	일반	메모리(tmpfs) 지향

공통점: 둘 다 키-값 맵이고 **env / 볼륨** 으로 Pod에 들어옴.

앱 입장: Secret의 Base64는 **kubelet**이 풀어서 넣어주므로, ConfigMap·Secret 둘 다 **평문**으로 도착 (앱은 같은 코드로 읽음).

어느 쪽을 쓸까 — 빠른 결정 가이드

데이터	권장
포트 번호, 기능 플래그, 로그 레벨	ConfigMap (env)
nginx/Java/prometheus 설정 파일	ConfigMap (볼륨)
패스워드, API 토큰	Secret (가능하면 파일 마운트)
TLS 인증서·개인키	Secret (<code>generic</code> 또는 <code>kubernetes.io/tls</code>)
프라이빗 레지스트리 자격 증명	Secret (<code>dockerconfigjson</code> + <code>imagePullSecrets</code>)
ServiceAccount 토큰	Secret (자동 생성·마운트)
깃에 커밋해도 OK한 설정	ConfigMap
깃에 커밋 안 됨	Secret + Sealed Secrets / Vault

규칙: "공개되면 사고가 나는가?" → 그렇다면 **Secret**, 아니면 **ConfigMap**

"Secret이면 안전"은 오해

- **etcd 저장**: 일반 객체와 같은 곳 → **etcd 암호화 at rest** 옵션 켜야 함
- **RBAC**: Secret 읽기 권한을 누구에게 주는가가 핵심
- **Pod 생성 권한**이 있으면 Secret을 마운트한 Pod로 우회 접근 가능
- **env로 Secret** 주입 시 디버깅 도구·로그·크래시 덤프에 노출 위험 → **파일 마운트 선호**
- **외부 비밀저장소**: Vault, 클라우드 KMS, External Secrets Operator 등과 결합

실습 9 — Secret 으로 HTTPS 인증서·키 마운트

```
# 1) ConfigMap (HTTPS용 nginx 설정)
$ kubectl create configmap fortune-config \
  --from-file=configmap-files/sleep-interval \
  --from-file=my-nginx-config.conf=https-nginx.conf

# 2) Secret (TLS 키 + 인증서)
$ kubectl create secret generic fortune-https \
  --from-file=fortune-https/https.key \
  --from-file=fortune-https/https.cert \
  --from-file=fortune-https/foo

$ kubectl apply -f fortune-pod-https.yaml
pod/fortune-https created

$ curl -skI https://127.0.0.1:18443/ | head -3
HTTP/1.1 200 OK
Server: nginx/1.29.8
```

역할 분리: 공개 설정 = ConfigMap, 키/인증서 = Secret → nginx가 Secret 마운트 경로 참조

마무리: 오늘의 세 줄

1. 분리하라 — Pod는 "어떻게 띄울지", ConfigMap/Secret은 "무엇을 넣을지"
2. 주입 경로 — 짧은 값은 env, 파일 기대 도구는 볼륨, 민감하면 Secret
3. 갱신은 즉시가 아니다 — 핫 리로드의 한계 → 버전드/롤링 패턴 검토